



TECNOLÓGICO
NACIONAL DE MÉXICO



**TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO SUPERIOR DE MISANTLA**

ASIGNATURA: Taller de Base de Datos

DOCENTE: Irahan Otoniel José Guzmán

NOMBRE DE LA ACTIVIDAD:

Reporte de practica "CRUD de diferentes situaciones"

UNIDAD No.: Unidad 2

CARRERA:

Ingeniería en Sistemas Computacionales

SEMESTRE: 05

GRUPO: 503 - S

ALUMNO: Jaret Isai Sanchez Jimenez

16 - 09 - 2026

Índice

Introducción	4
Inicialización de entorno	5
TABLA DE COMIDA.....	5
Ejercicio 1	5
CREATE comida.....	6
INSERT comida	6
UPDATE comida.....	7
DETELE comida	7
TABLA DE ALUMNOS	7
Ejercicio 2	7
CREATE alumnos.....	8
INSERT alumnos	9
UPDATE alumnos.....	9
DELETE alumnos	9
TABLA DE PRODUCTOS	10
Ejercicio 3	10
CREATE productos	10
INSERT productos.....	11
UPDATE productos	11
DELETE productos.....	12
Conclusión	13

Índice de figuras

Fig. 1 Inicialización del entorno	5
Fig. 2 Datos del ejercicio 1	5
Fig. 3 Create comida	6
Fig. 4 Insert comida	6
Fig. 5 Update comida	7
Fig. 6 Delete comida	7
Fig. 7 Datos del ejercicio 2	8
Fig. 8 Create alumnos	8
Fig. 9 Insert alumnos	9
Fig. 10 Update alumnos	9
Fig. 11 Delete alumnos	10
Fig. 12 Datos del ejercicio 3	10
Fig. 13 Create productos	11
Fig. 14 Insert productos	11
Fig. 15 Update productos	12
Fig. 16 Delete productos	12

Introducción

El presente reporte documenta la implementación de scripts en Python orientados a la gestión de bases de datos relacionales mediante la biblioteca sqlite3. A través de tres ejercicios prácticos, se analiza el comportamiento de las instrucciones DDL (Lenguaje de Definición de Datos) y DML (Lenguaje de Manipulación de Datos) para crear tablas, insertar registros, actualizar valores y eliminar datos (operaciones CRUD). Se utiliza pandas para renderizar los resultados de las consultas en forma de *DataFrames* para su fácil lectura.

Inicialización de entorno

Se utiliza en los siguientes 3 ejercicios. Primero se importan los módulos `sqlite3` y `pandas`, que permiten trabajar con la base de datos y mostrar los resultados de las consultas de una forma más ordenada. Se crea una conexión a una base de datos SQLite en memoria utilizando `sqlite3.connect(":memory:")`. Este tipo de base de datos es temporal, ya que solo existe mientras el programa está en ejecución. También se utilizan las funciones `query()` y `run()`: la primera sirve para realizar consultas `SELECT` y mostrar los resultados en un `DataFrame`, mientras que la segunda permite ejecutar instrucciones como `INSERT`, `UPDATE`, `DELETE` y `DDL`, guardando los cambios con `conn.commit()`.

```
import sqlite3
import pandas as pd

# Conexión en memoria (se pierde al cerrar el notebook)
conn = sqlite3.connect(":memory:")
cur = conn.cursor() #objeto para ejecutar comandos de SQL

def query(sql, params=()):
    """Ejecuta un SELECT y devuelve un DataFrame para verlo bonito."""
    return pd.read_sql_query(sql, conn, params=params)

def run(sql, params=()):
    """Ejecuta INSERT/UPDATE/DELETE/DDl y hace commit."""
    cur.execute(sql, params)
    conn.commit()
    print("OK ->", sql.strip().splitlines()[0], "...")
```

Fig. 1 Inicialización del entorno

TABLA DE COMIDA

Ejercicio 1

`CREATE TABLE` (Realizar un `INSERT`, `UPDATE` Y `DELETE`). Una cadena de restaurantes necesita registrar en su base de datos el menú de platillos que ofrece. Crea una tabla llamada `comida` que cumpla con las siguientes especificaciones:

Campo	Tipo de dato	Restricción
<code>id_platillo</code>	<code>INT</code>	Llave primaria (PK), autoincremental
<code>nombre</code>	<code>VARCHAR(50)</code>	No puede ser nulo
<code>categoria</code>	<code>VARCHAR(30)</code>	Ej: entrada, plato fuerte, postre, bebida
<code>precio</code>	<code>DECIMAL(6,2)</code>	No puede ser nulo
<code>disponible</code>	<code>BOOLEAN</code>	Valor por defecto: <code>TRUE</code>

Fig. 2 Datos del ejercicio 1

En este enlace se muestran los resultados obtenidos en Google colab del ejercicio 1:[**ejercicio 1 - Colab**](#)

CREATE comida

Primero se utiliza DROP TABLE IF EXISTS comida; para eliminar cualquier versión anterior de la tabla. Después se crea la tabla comida con los campos necesarios para registrar los platillos. Se incluye id_platillo como clave primaria, además de nombre, categoria, precio y disponible. El campo disponible tiene un valor por defecto de true, que en SQLite se representa como 1, por lo que los platillos quedan disponibles automáticamente al ser registrados.

```
cur.executescript('''
DROP TABLE IF EXISTS comida;

CREATE TABLE comida (
    id_platillo integer PRIMARY KEY,
    nombre varchar(50) NOT NULL,
    categoria varchar(30) NOT NULL,
    precio decimal(6,2) NOT NULL,
    disponible boolean default true
);

''')
conn.commit()
print("Tablas creadas: comida")

... Tablas creadas: comida
```

Fig. 3 Create comida

INSERT comida

Se agregan tres platillos al menú: coctel de camaron, de categoría ligero, con un precio de \$60.00; caldo de pollo, de categoría fuerte, con un precio de \$95.00; y sopa fria, de categoría entrada, con un precio de \$30.00. Como no se especifica un valor para disponible, se utiliza automáticamente el valor establecido por defecto. Después se consulta la tabla para comprobar que los registros fueron agregados correctamente.

```
# INSERT: alta de comida
run("""INSERT INTO comida (nombre, categoria, precio) VALUES
('coctel de camaron','ligero', 60.00),
('caldo de pollo', 'fuerte', 95.00),
('sopa fria', 'entrada', 30.00);""")

query("SELECT * FROM comida")

OK -> INSERT INTO comida (nombre, categoria, precio) VALUES ...
id_platillo      nombre categoria precio disponible
0                1 coctel de camaron  ligero    60         1
1                2 caldo de pollo   fuerte    95         1
```

Fig. 4 Insert comida

UPDATE comida

En esta etapa se aplica un aumento del 10% al precio de los platillos pertenecientes a la categoría fuerte. Para ello se utiliza UPDATE comida SET precio = precio * 1.10 WHERE categoria = 'fuerte';. Como resultado, el caldo de pollo pasa de \$95.00 a \$104.50. Después se realiza una consulta para comprobar que la modificación se realizó correctamente.

```
# UPDATE: modificación de registros (aumento del precio de comidas fuertes)
run("UPDATE comida SET precio = precio * 1.10 WHERE categoria = 'fuerte';")
query("SELECT id_platillo, nombre, categoria, precio FROM comida WHERE categoria = 'fuerte'")

OK -> UPDATE comida SET precio = precio * 1.10 WHERE categoria = 'fuerte'; ...
```

	id_platillo	nombre	categoria	precio
0	2	caldo de pollo	fuerte	104.5

Fig. 5 Update comida

DELETE comida

Finalmente, se elimina el platillo sopa fria utilizando DELETE FROM comida WHERE nombre = 'sopa fria';. Para comprobar el resultado, se consulta nuevamente la tabla completa. Después de la eliminación, únicamente permanecen los registros correspondientes al coctel de camaron y al caldo de pollo.

```
# DELETE: eliminación de registros (se retira una comida)
run("DELETE FROM comida WHERE nombre = 'sopa fria';")
query("SELECT * FROM comida")

OK -> DELETE FROM comida WHERE nombre = 'sopa fria'; ...
```

	id_platillo	nombre	categoria	precio	disponible
0	1	coctel de camaron	ligero	60.0	1
1	2	caldo de pollo	fuerte	104.5	1

Fig. 6 Delete comida

TABLA DE ALUMNOS

Ejercicio 2

Escuela (tabla alumnos) (Realizar CREATE, INSERT, UPDATE Y DELETE) Una escuela quiere registrar la información básica de sus alumnos.

Campo	Tipo de dato	Restricción
id_alumno	INT	Llave primaria (PK), autoincremental
nombre	VARCHAR(50)	No puede ser nulo
grado	VARCHAR(10)	Ej: "3ro A", "5to B"
edad	INT	No puede ser nulo
promedio	DECIMAL(4,2)	Valor por defecto: 0.0

Fig. 7 Datos del ejercicio 2

En este enlace se muestran los resultados obtenidos en Google colab del ejercicio 2: [ejercicio 2 - Colab](#)

CREATE alumnos

Primero se ejecuta DROP TABLE IF EXISTS alumnos; para evitar problemas si la tabla ya había sido creada anteriormente. Después se crea la tabla alumnos, donde id_alumno funciona como clave primaria. También se agregan los campos nombre, grado, edad y promedio. El campo edad se establece como entero y obligatorio, mientras que promedio permite almacenar las calificaciones con dos decimales.

```
cur.executescript('''
DROP TABLE IF EXISTS alumnos;

CREATE TABLE alumnos (
    id_alumno integer PRIMARY KEY,
    nombre varchar(50) NOT NULL,
    grado varchar(10) NOT NULL,
    edad integer NOT NULL,
    promedio decimal(4,2)
);

''')
conn.commit()
print("Tablas creadas: alumnos")
```

Fig. 8 Create alumnos

INSERT alumnos

En esta parte se agregan tres alumnos a la tabla. Los registros corresponden a Juan, quien cursa 5to A, tiene 15 años y un promedio de 7.8; Pedro, de 3ro B, con 13 años y un promedio de 6.9; y Maria, de 2do A, con 14 años y un promedio de 9.0. Después de realizar las inserciones, se ejecuta una consulta para comprobar que los datos fueron registrados correctamente.

```
# INSERT: alta de alumnos
run("""INSERT INTO alumnos (nombre, grado, edad, promedio) VALUES
('Juan', '5to A', 15, 7.8),
('Pedro', '3ro B', 13, 6.9),
('Maria', '2do A', 14, 9.0);""")

query("SELECT * FROM alumnos")

... OK -> INSERT INTO alumnos (nombre, grado, edad, promedio) VALUES ...
```

	id_alumno	nombre	grado	edad	promedio
0	1	Juan	5to A	15	7.8
1	2	Pedro	3ro B	13	6.9
2	3	Maria	2do A	14	9.0

Fig. 9 Insert alumnos

UPDATE alumnos

Se modifica el promedio del alumno que pertenece al grupo 3ro B mediante la instrucción UPDATE alumnos SET promedio = promedio * 1.10 WHERE grado = '3ro B';. De esta manera, el promedio de Pedro aumenta de 6.9 a 7.59. Después se realiza una consulta utilizando WHERE grado = 'fuerte'. Como ningún alumno pertenece a ese grado, la consulta no devuelve registros y el resultado aparece vacío.

```
# UPDATE: modificación de registros (aumento del promedio de alumnos 3ro B)
run("UPDATE alumnos SET promedio = promedio * 1.10 WHERE grado = '3ro B';")
query("SELECT id_alumno, nombre, grado, edad, promedio FROM alumnos WHERE grado = 'fuerte'")

OK -> UPDATE alumnos SET promedio = promedio * 1.10 WHERE grado = '3ro B'; ...
```

	id_alumno	nombre	grado	edad	promedio
--	-----------	--------	-------	------	----------

Fig. 10 Update alumnos

DELETE alumnos

Para finalizar, se elimina el registro correspondiente a Juan mediante DELETE FROM alumnos WHERE nombre = 'Juan';. Después se muestra nuevamente la tabla para comprobar el resultado. Al terminar la operación, permanecen únicamente los registros de Pedro y Maria.

```

# DELETE: eliminación de registros (se retira un alumno)
run("DELETE FROM alumnos WHERE nombre = 'Juan';")
query("SELECT * FROM alumnos")

... OK -> DELETE FROM alumnos WHERE nombre = 'Juan'; ...

  id_alumno  nombre  grado  edad  promedio
0          2   Pedro   3ro B   13      7.59
1          3    Maria   2do A   14      9.00

```

Fig. 11 Delete alumnos

TABLA DE PRODUCTOS

Ejercicio 3

Tiendita escolar (tabla productos) (Realizar CREATE, INSERT, UPDATE Y DELETE)
La tiendita de la escuela necesita llevar el control de los productos que vende.

Campo	Tipo de dato	Restricción
id_producto	INT	Llave primaria (PK), autoincremental
nombre	VARCHAR(50)	No puede ser nulo
tipo	VARCHAR(30)	Ej: snack, bebida, dulce, papelería
precio	DECIMAL(6,2)	No puede ser nulo
existencia	INT	Valor por defecto: 0

Fig. 12 Datos del ejercicio 3

En este enlace se muestran los resultados obtenidos en Google colab del ejercicio 3: [ejercicio 3 - Colab](#)

CREATE productos

Para comenzar con una tabla limpia, se ejecuta DROP TABLE IF EXISTS productos;, lo que elimina la tabla en caso de que ya exista. Después se crea la tabla productos mediante CREATE TABLE. En ella, id_productos funciona como clave primaria y se genera automáticamente, mientras que nombre y tipo son campos obligatorios. También se agregan los campos precio para almacenar el costo del producto y existencia para guardar la cantidad disponible.

```

cur.executescript('''
DROP TABLE IF EXISTS productos;

CREATE TABLE productos (
    id_productos integer PRIMARY KEY,
    nombre varchar(50) NOT NULL,
    tipo varchar(30) NOT NULL,
    precio decimal(6,2) NOT NULL,
    existencia integer
);

''')
conn.commit()
print("Tablas creadas: productos")

Tablas creadas: productos

```

Fig. 13 Create productos

INSERT productos

Después de crear la tabla, se agregan tres productos: jugos, de tipo beber, con un precio de \$20.00; chocolates, de tipo golosina, con un precio de \$13.00; y sabritas, de tipo botana, con un precio de \$15.00. En este caso no se proporciona un valor para el campo existencia, por lo que SQLite lo almacena como NULL. Finalmente, se utiliza una consulta SELECT para comprobar que los tres registros fueron agregados correctamente.

```

# INSERT: alta de productos
run("""INSERT INTO productos (nombre, tipo, precio) VALUES
('jugos','beber', 20.00),
('chocolates', 'golosina', 13.00),
('sabritas', 'botana', 15.00);""")

query("SELECT * FROM productos")

*** OK -> INSERT INTO productos (nombre, tipo, precio) VALUES ...

```

	id_productos	nombre	tipo	precio	existencia
0	1	jugos	beber	20	None
1	2	chocolates	golosina	13	None
2	3	sabritas	botana	15	None

Fig. 14 Insert productos

UPDATE productos

Para modificar información de la tabla se aplica un aumento del 10% al precio de los productos cuya categoría sea golosina. Esto se realiza con la sentencia UPDATE productos SET precio = precio * 1.10 WHERE tipo = 'golosina';. Como resultado, el precio de chocolates cambia de \$13.00 a \$14.30. Después se realiza una consulta para comprobar que el cambio se aplicó correctamente.

```
# UPDATE: modificación de registros (aumento del precio de productos golosina)
run("UPDATE productos SET precio = precio * 1.10 WHERE tipo = 'golosina'");
query("SELECT id_productos, nombre, tipo, precio FROM productos WHERE tipo = 'golosina'")

OK -> UPDATE productos SET precio = precio * 1.10 WHERE tipo = 'golosina'; ...
      id_productos  nombre  tipo  precio
0          2  chocolates  golosina  14.3
```

Fig. 15 Update productos

DELETE productos

Por último, se elimina el producto sabritas utilizando la instrucción DELETE FROM productos WHERE nombre = 'sabritas';. Después se consulta nuevamente la tabla para verificar el resultado. Al finalizar, solamente permanecen los productos jugos y chocolates.

```
# DELETE: eliminación de registros (se retira unos productos)
run("DELETE FROM productos WHERE nombre = 'sabritas'");
query("SELECT * FROM productos")

... OK -> DELETE FROM productos WHERE nombre = 'sabritas'; ...
      id_productos  nombre  tipo  precio  existencia
0          1      jugos  beber  20.0      None
```

Fig. 16 Delete productos

Conclusión

La segmentación visual y lógica de las operaciones CRUD en estos ejercicios permite comprender a profundidad el ciclo de vida de los datos. Desde la definición estructural con CREATE, la alimentación de la base con INSERT, la manipulación de información existente con restricciones específicas mediante UPDATE, hasta la depuración de datos con DELETE.

Dominar la sintaxis de SQL y su integración con lenguajes de alto nivel como Python es esencial en la Ingeniería en Sistemas Computacionales. Las funciones abstractas creadas (run y query) demuestran cómo se puede modularizar el código para interactuar con bases de datos de forma segura (aplicando commit) y cómo librerías externas (Pandas) mejoran sustancialmente la experiencia del usuario y del desarrollador al visualizar resultados estructurados.